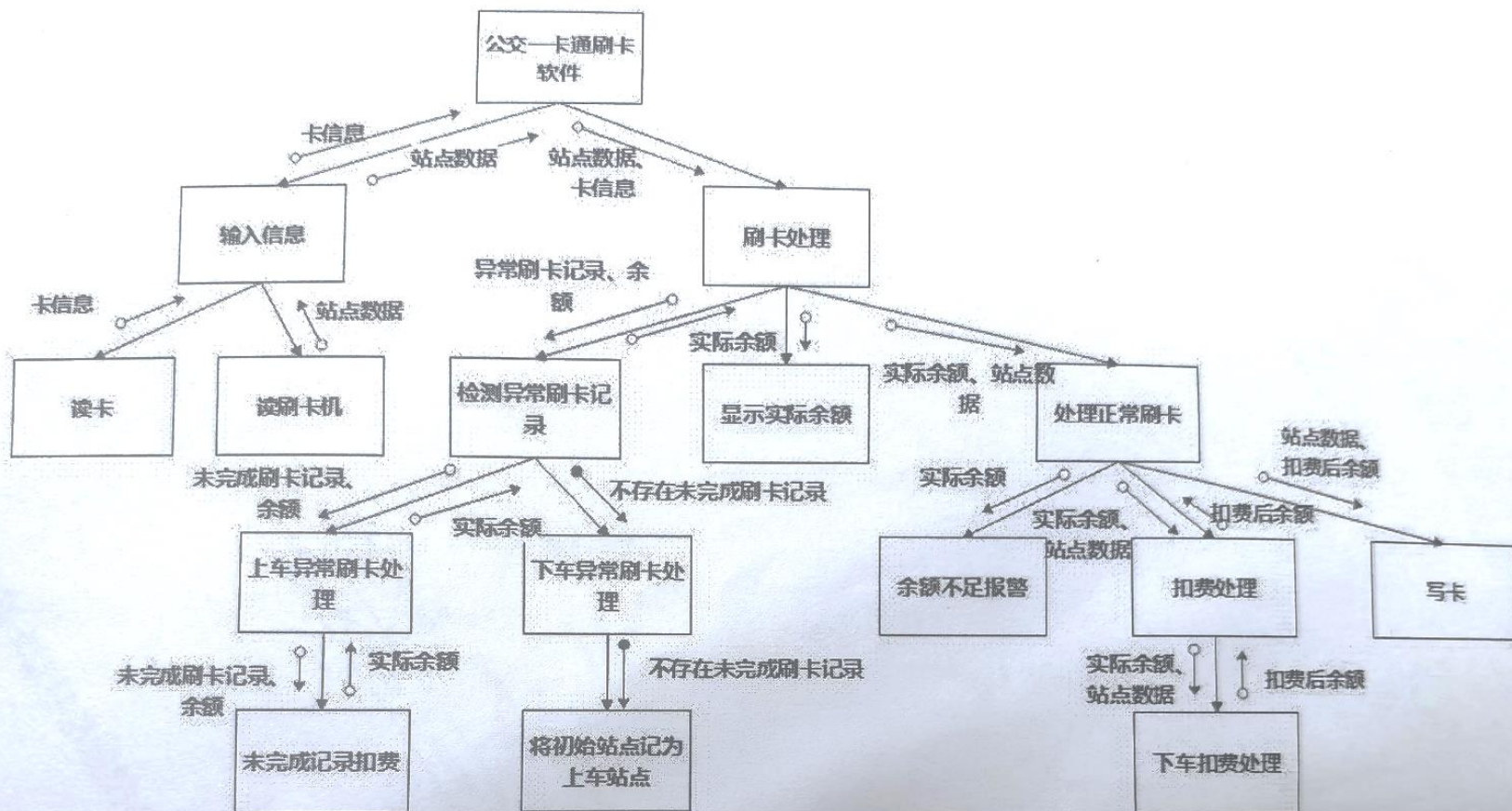
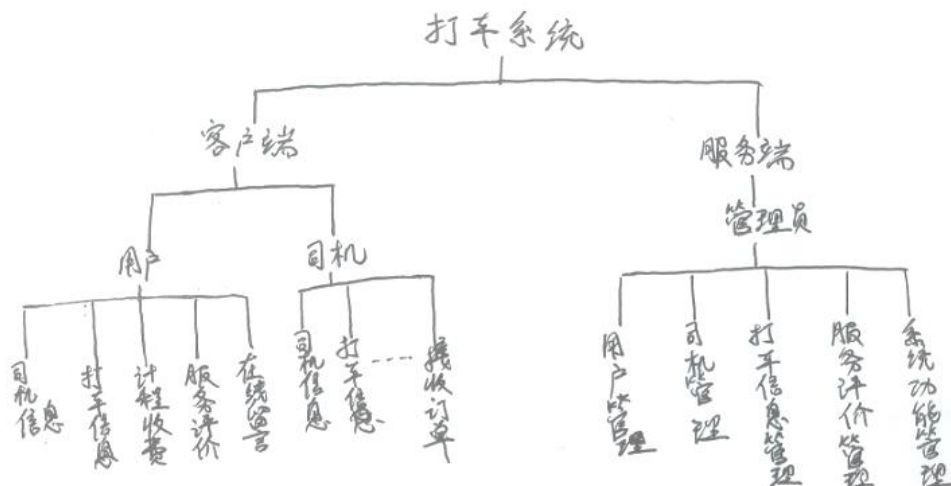


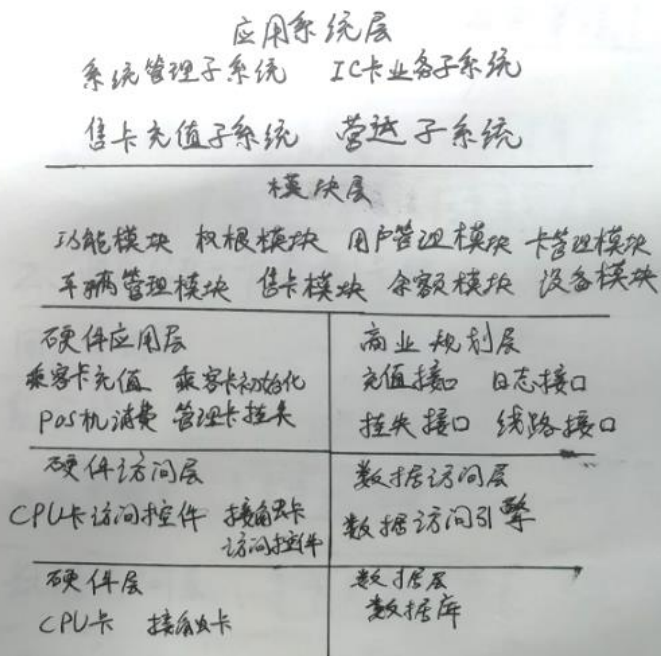
第八章 面向对象设计



手机打车软件系统结构图



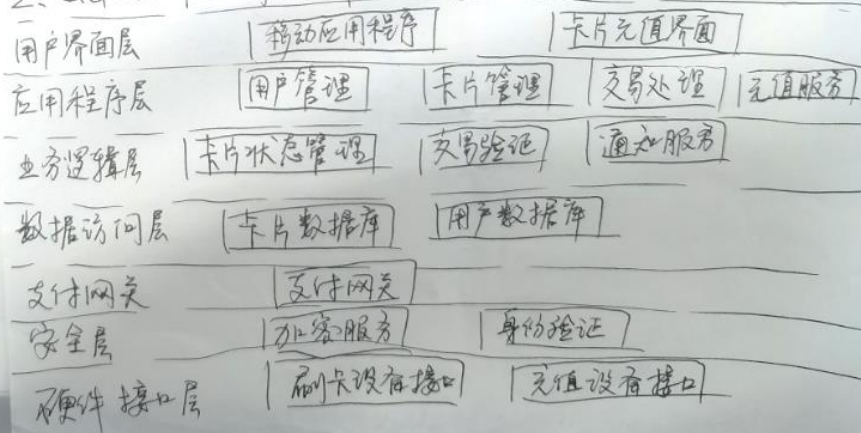
公交车一卡通刷卡软件系统结构图



1. 手机打车软件的系统结构图



2. 画出公交一卡通刷卡软件的系统结构图



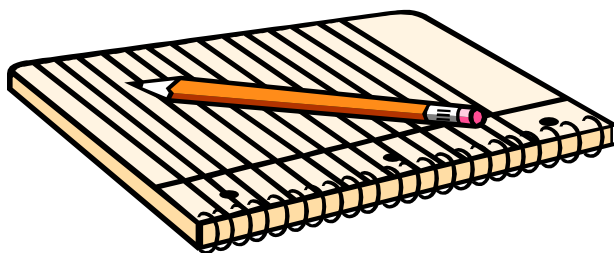
本章要点

- 面向对象设计与UML

- 类图与对象图

- 顺序图和协作图

- 实现图



8.1 面向对象设计与UML

- UML的重要内容可以由以下五类共九种图来定义，用例图、行为图和交互图用来对软件行为建模，静态图 and 实现图用来对软件结构建模。

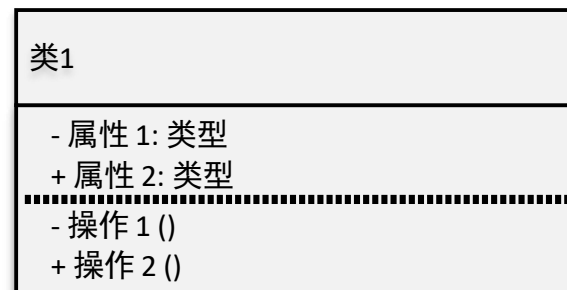
建模对象	图类型	UML图
软件行为	用例图	用例图
	行为图	活动图、状态图
	交互图	顺序图、协作图
软件结构	静态图	类图、对象图
	实现图	构件图、部署图

UML图

- 在需求获取和分析阶段，常常用用例图获取用户的功能需求，用活动图和状态图对业务流程建模。
- 在系统设计阶段，活动图和状态图也被用来对系统流程建模。
- 交互图描述对象间的交互关系，包括顺序图和协作图。
- 静态图是从系统内部来看数据和功能性，主要包括类图、对象图和包图。
- 实现图是将系统的结构和行为表达成为易于转换为实现的方式，包括构件图和部署图。

8.2 类图和对象图

- 8.2.1 类与类图
- 在UML中，类通过一个矩形表示，被两条直线分隔成3个部分，分别表示类名、属性和方法。属性和方法之前可附加的可见性修饰符：
 - 加号（+）表示public；
 - 减号（-）表示private；
 - 井号（#）表示protected；

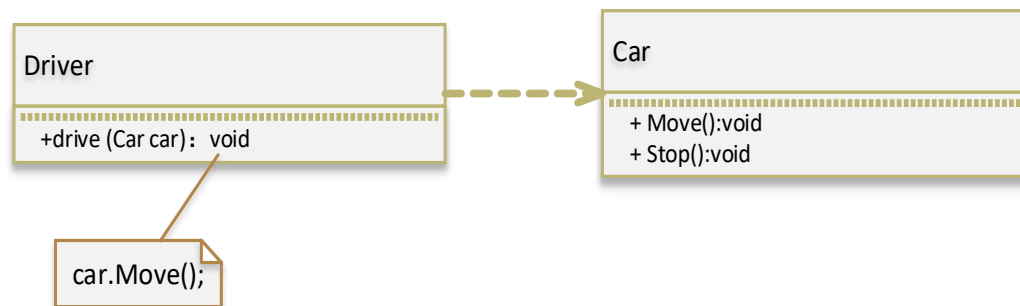


8.2.2 类之间关系

- 在UML类图中，常见的有以下几种关系：泛化（Generalization），实现（Realization），关联（Association），聚合（Aggregation），组合（Composition），依赖（Dependency）。
- 根据类与类之间的耦合度从弱到强排列分别是：依赖关系、关联关系、聚合关系、组合关系、泛化关系和实现关系。其中泛化和实现的耦合度相等，它们是最强的。

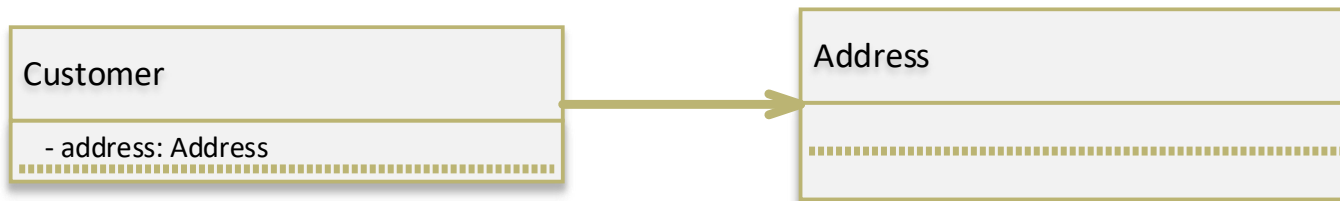
8.2.2 类之间关系

- 1. 依赖关系
- 依赖（Dependency）关系是一种使用关系，它是对象之间耦合度最弱的一种关联方式，是临时性的关联。
- 在UML中依赖关系使用虚线箭头连接两个类来表示，箭头从使用类指向被依赖的类



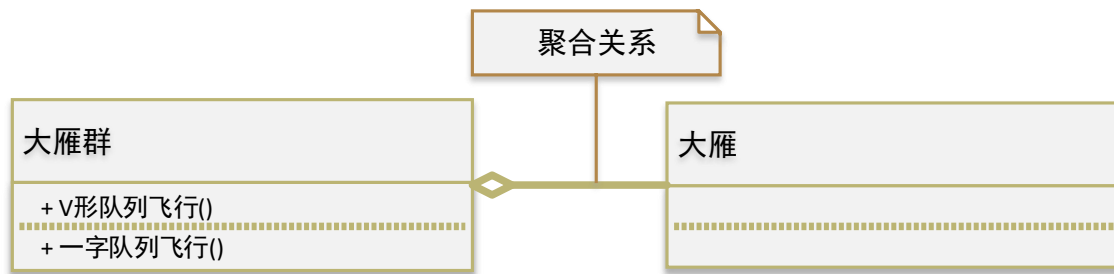
8.2.2 类之间关系

- 2. 关联关系
- 关联（Association）关系是对象之间的一种引用关系，在代码中通常将一个类的对象作为另一个类的成员变量来实现关联关系。
- 单向的关联用带一个箭头的实线来表示，箭头从使用类指向被关联的类。



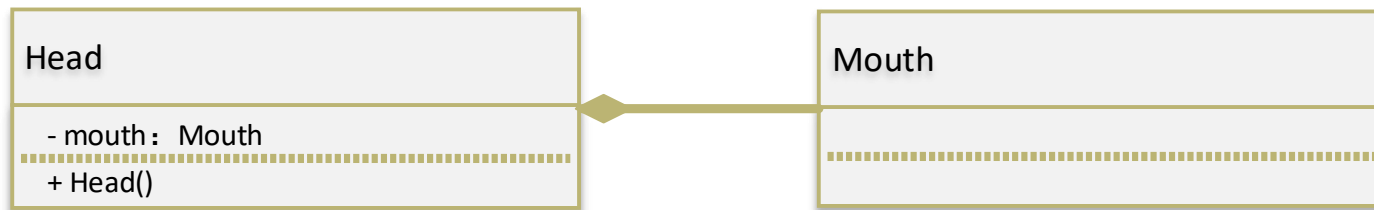
8.2.2 类之间关系

- 3. 聚合关系
- 聚合（Aggregation）关系是关联关系的一种，是强关联关系，是整体和部分之间的关系，是 has-a 的关系。成员对象是整体对象的一部分，但是成员对象可以脱离整体对象而独立存在
- 在 UML 类图中，聚合关系可以用带空心菱形的实线来表示，菱形指向整体。



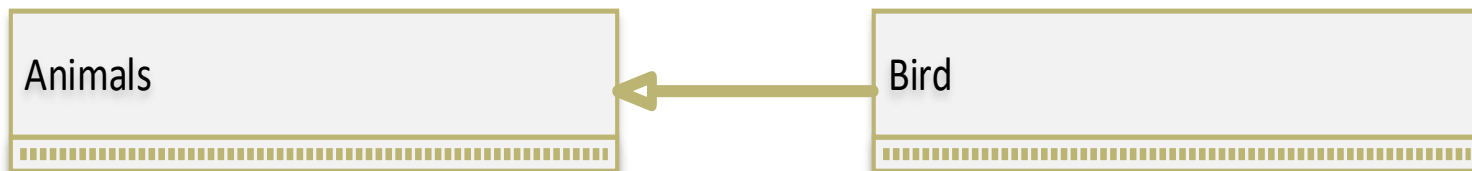
8.2.2 类之间关系

- 4. 组合关系
- 组合（Composition）关系也是关联关系的一种，也表示类之间的整体与部分的关系，但它是一种更强烈的聚合关系。
- 在组合关系中，整体对象可以控制部分对象的生命周期，一旦整体对象不存在，部分对象也将不存在，部分对象不能脱离整体对象而存在。例如，头和嘴的关系，没有了头，嘴也就不存在了。在 UML 类图中，组合关系用带实心菱形的实线来表示，菱形指向整体。



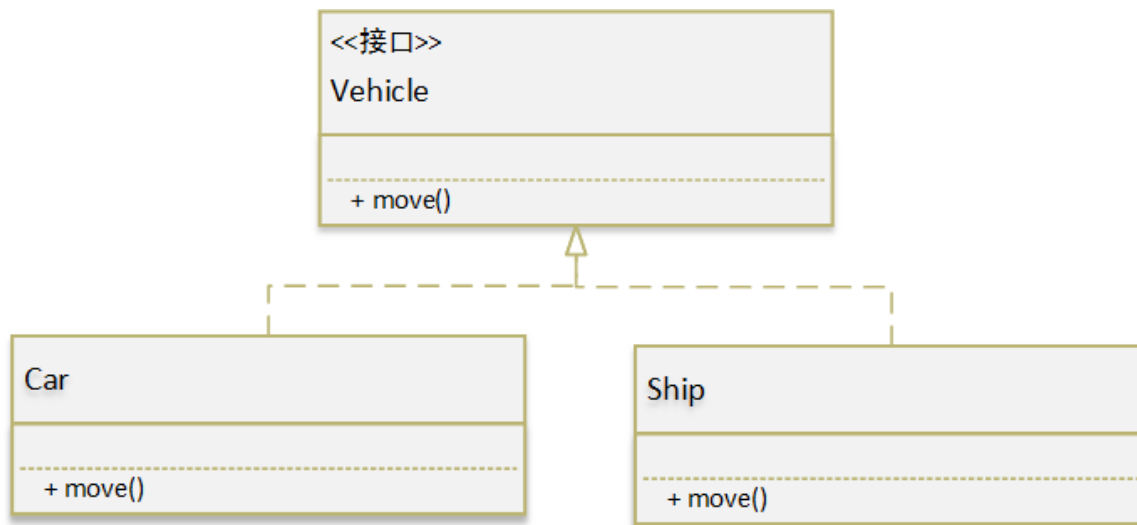
8.2.2 类之间关系

- 5. 泛化 (Generalization)
- 泛化关系是一种继承关系，表示一般与特殊的关系，它指定了子类如何特殊化父类的所有特征和行为。
- 泛化关系是带三角箭头的实线，箭头指向父类。



8.2.2 类之间关系

- 6. 实现 (Realization)
- 实现关系是一种类与接口的关系，表示类是接口所有特征和行为的实现，在这种关系中，类实现了接口，类中的操作实现了接口中所声明的所有的抽象操作。
- 在 UML 类图中实现关系用带空心三角形的虚线表示，箭头指向接口。





8.2.3 对象图

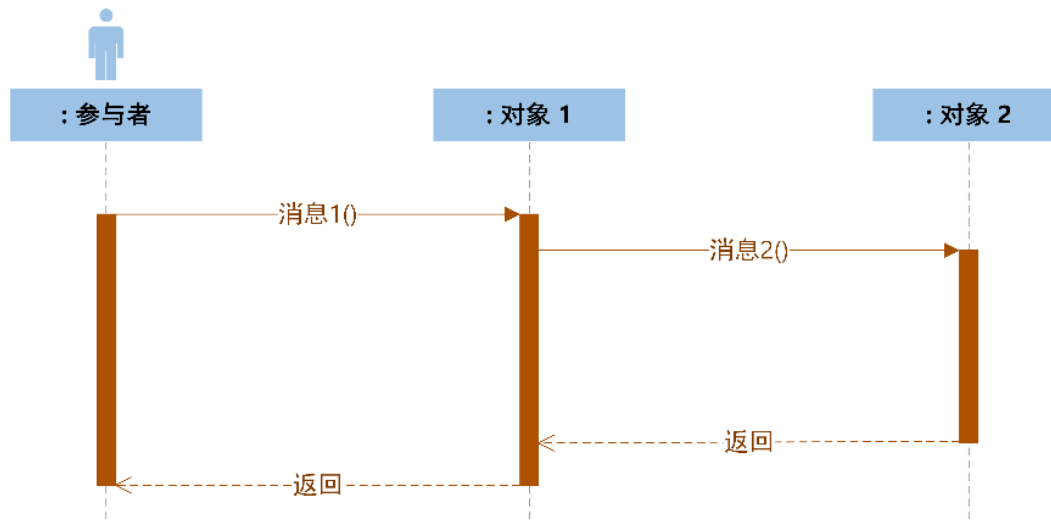
- 对象图显示了某一时刻的一组对象及它们之间的关系。
- 一个UML对象图可看成一个类图的特殊用例，实例和类可在其中显示。
- 对象图几乎使用与类图完全相同的标识。

8.3 顺序图和协作图

- 交互图描述对象间的交互关系，包括顺序图和协作图
 - 顺序图（或称时序图）显示对象之间的动态合作关系，它**强调对象之间消息发送的顺序**，同时显示对象之间的交互，**强调时间和顺序**。
 - 协作图（或称合作图）**描述对象间的协作关系**，协作图跟顺序图相似，显示对象间的动态合作关系。除显示信息交换外，协作图还显示对象以及它们之间的关系。**协作图强调上下级关系**。

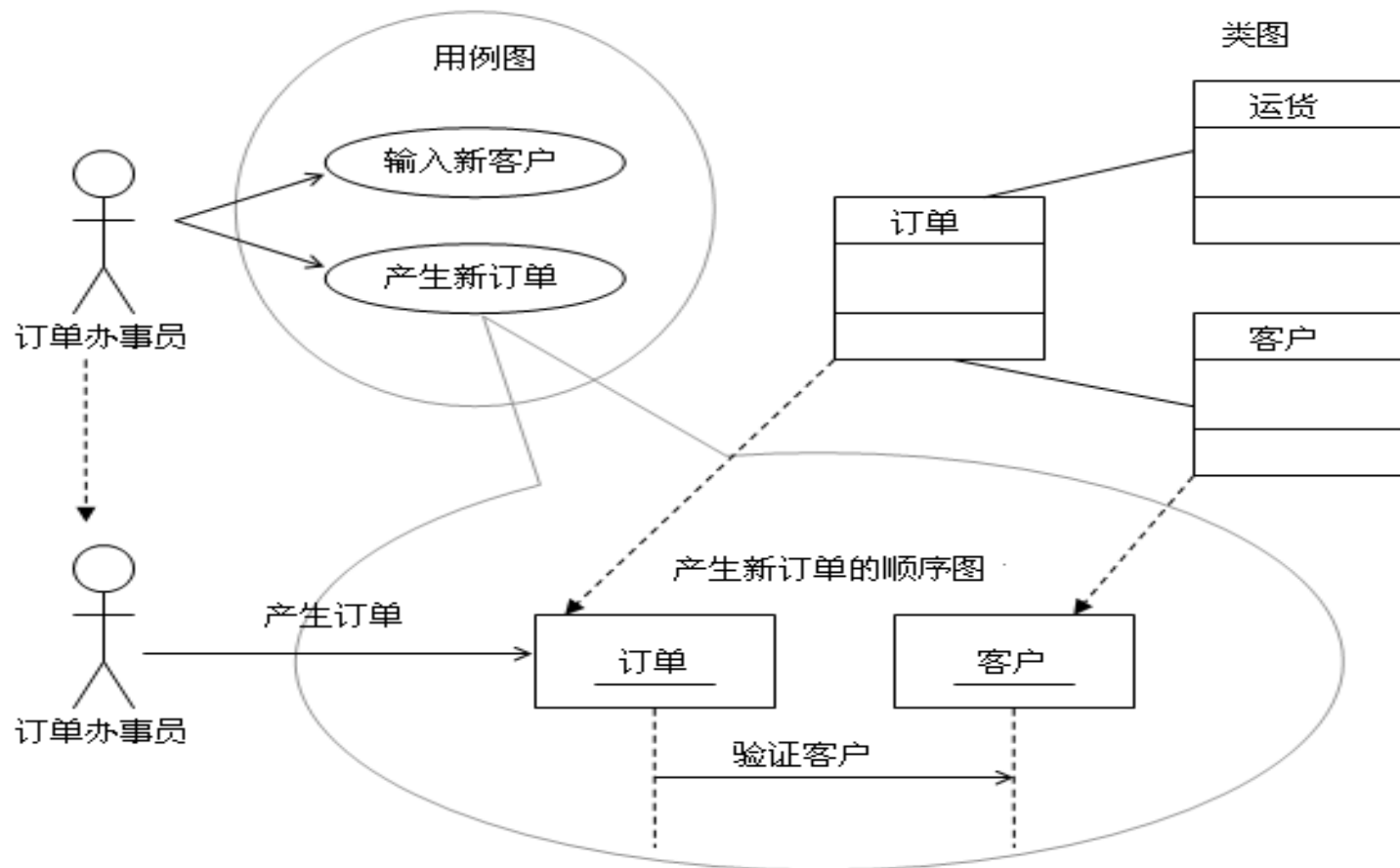
8.3.1 顺序图

- 顺序图把用例行为分配给类(对象), 并将交互关系表示为一个二维图, 纵向是时间轴, 时间沿竖线向下延伸, 横轴代表了在交互的参与者和独立类、对象或其组成的系统。



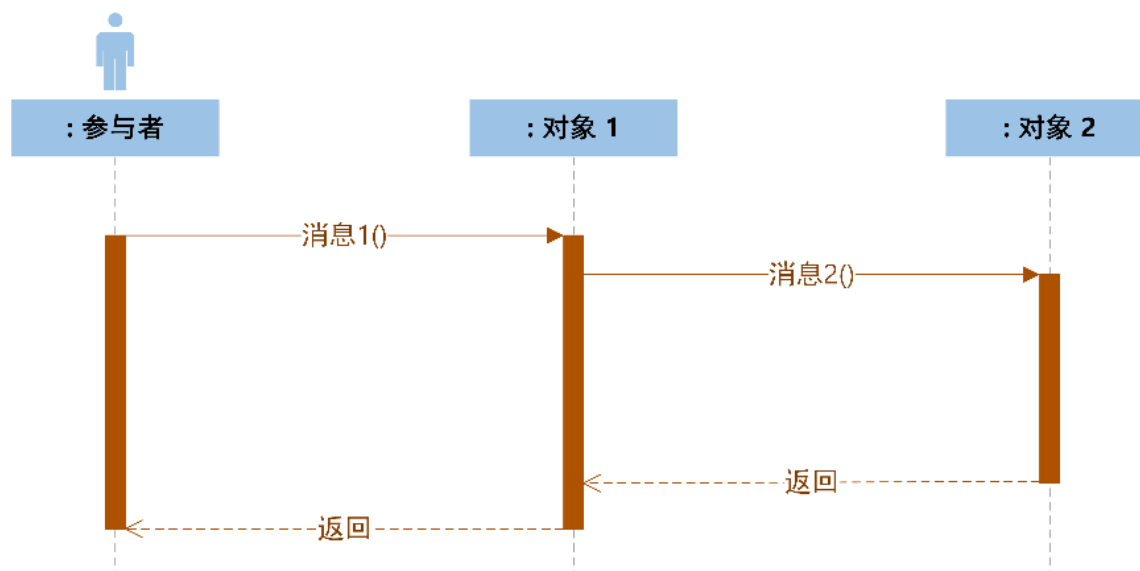
- 顺序图有两个主要的标记符: 活动对象和消息。

顺序图与用例图和类图的关系



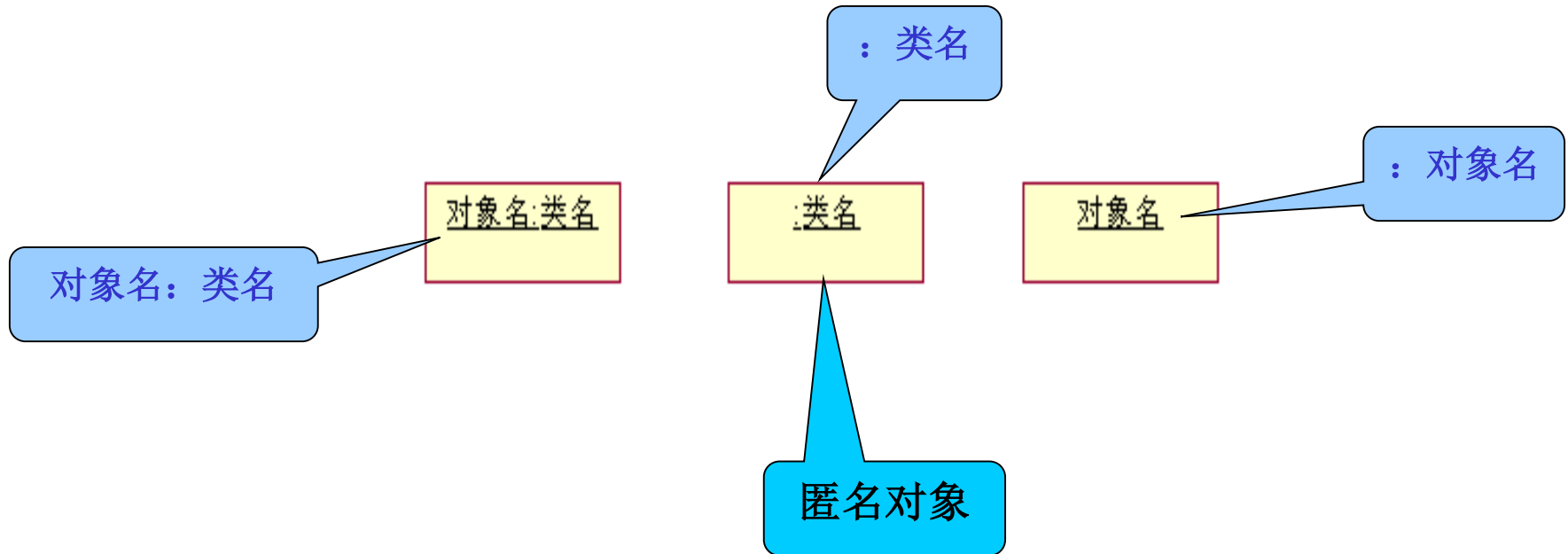
1. 活动对象

- 活动对象可以是系统的参与者或者任何有效的系统对象。对象是类的实例，它使用包围名称的矩形框来标记。使用“对象名：类名”的形式，把参与者表示为活动对象可以说明参与者如何与系统交互，以及系统如何与用户交互。



对象的命名和排序

(1) 对象的命名



对象的左右排列位置:

对象的左右顺序并不重要，但是为了图面的清晰整洁起见，通常应遵循以下两个原则：

- (1) 把交互频繁的对象尽可能地靠拢
- (2) 把初始化整个交互活动的对象（有时是一个参与者）放置在最左边

生命线

② 生命线

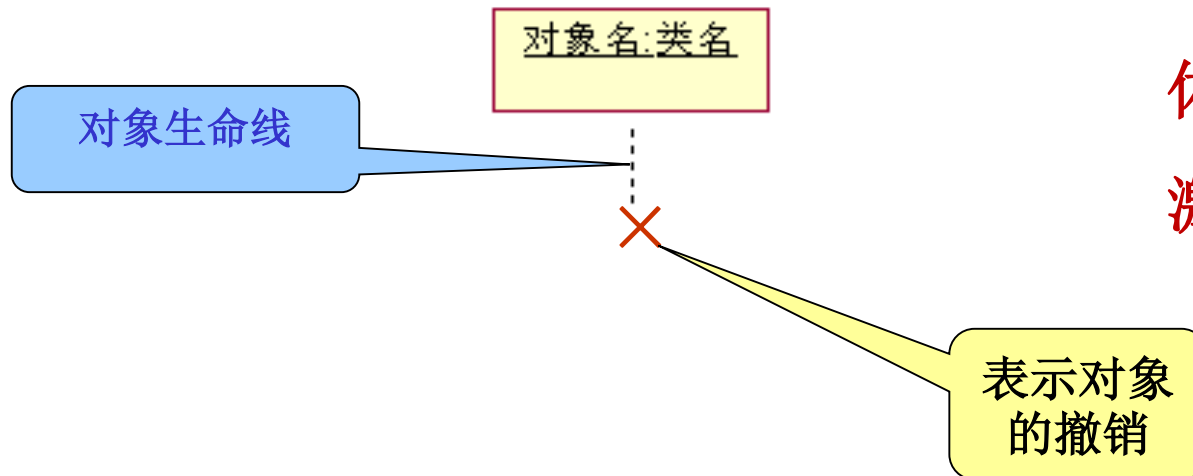
表示对象存在的时间，对象下面一条虚线表示。

生命线从对象创建开始到对象销毁时终止。

对象在生命线上的两种状态

休眠状态

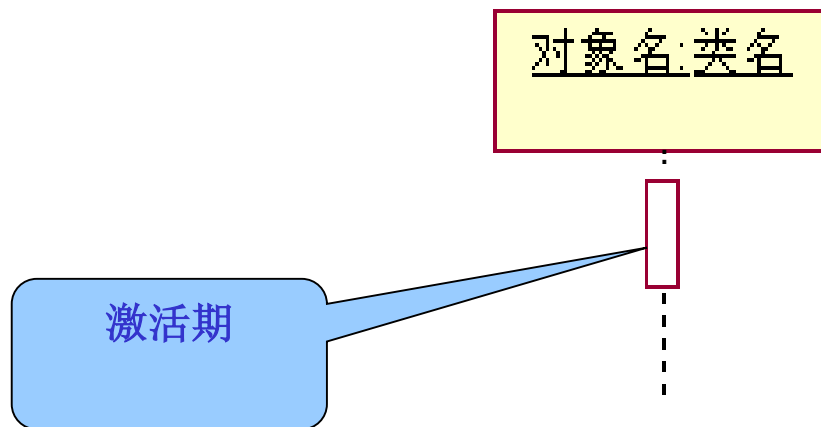
激活状态



控制焦点 / 激活期

③ 控制焦点 / 激活期

控制焦点/激活期用生命线上的小矩形表示，**表示这个时间内对象将执行操作**。矩形框的高度表示对象执行一个操作所经历的时间段，矩形的顶部表示动作的开始，底部表示动作的结束。**对象接收消息后可以由自己的某个操作来完成，也可以通过其他对象的操作来完成。**



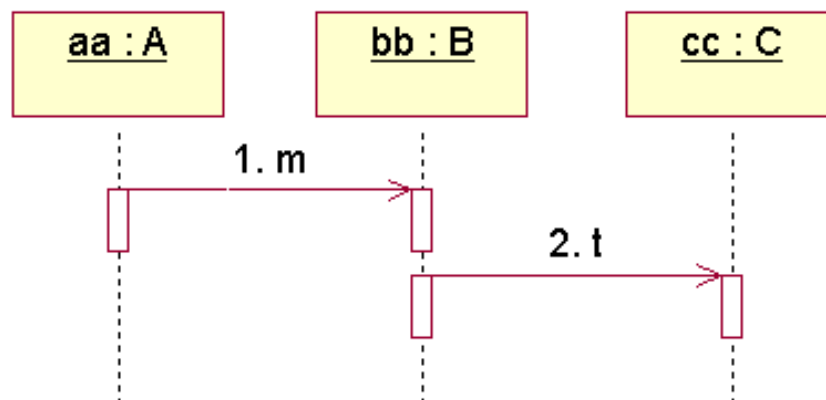
2. 消息

1 **消息**用来说明顺序图中不同活动对象之间的通信。

➤用从活动对象生命线到接收对象生命线的箭头表示消息。

箭头上标记要发送的消息，如下图所示。

➤消息一旦发送便将控制从源对象转移到目标对象。因为消息的阅读顺序是严格自上而下的，**因此活动对象之间发送的消息是顺序图的关键。**



消息的类型

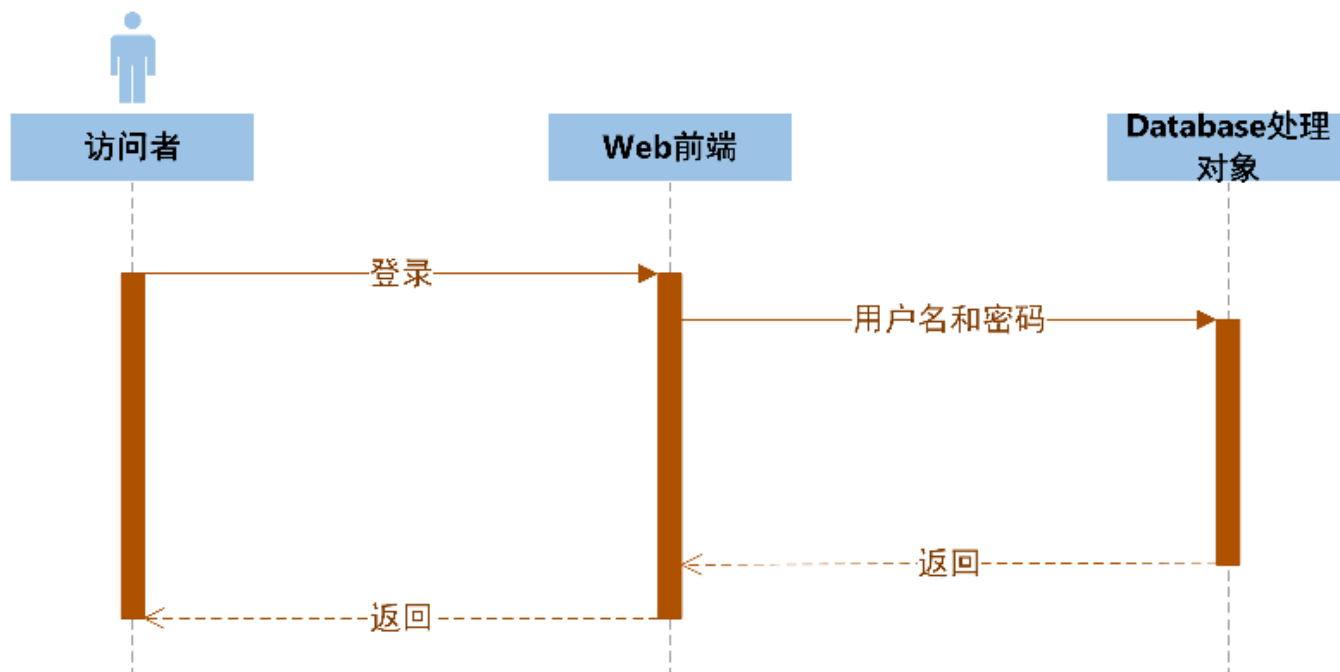
●在UML中，总共有4种类型的消息，如下图所示。

到目前为止只看到了一种消息，即简单消息（flat message）。

消息类型	图形
同步消息	
异步消息	
返回消息	
简单消息	

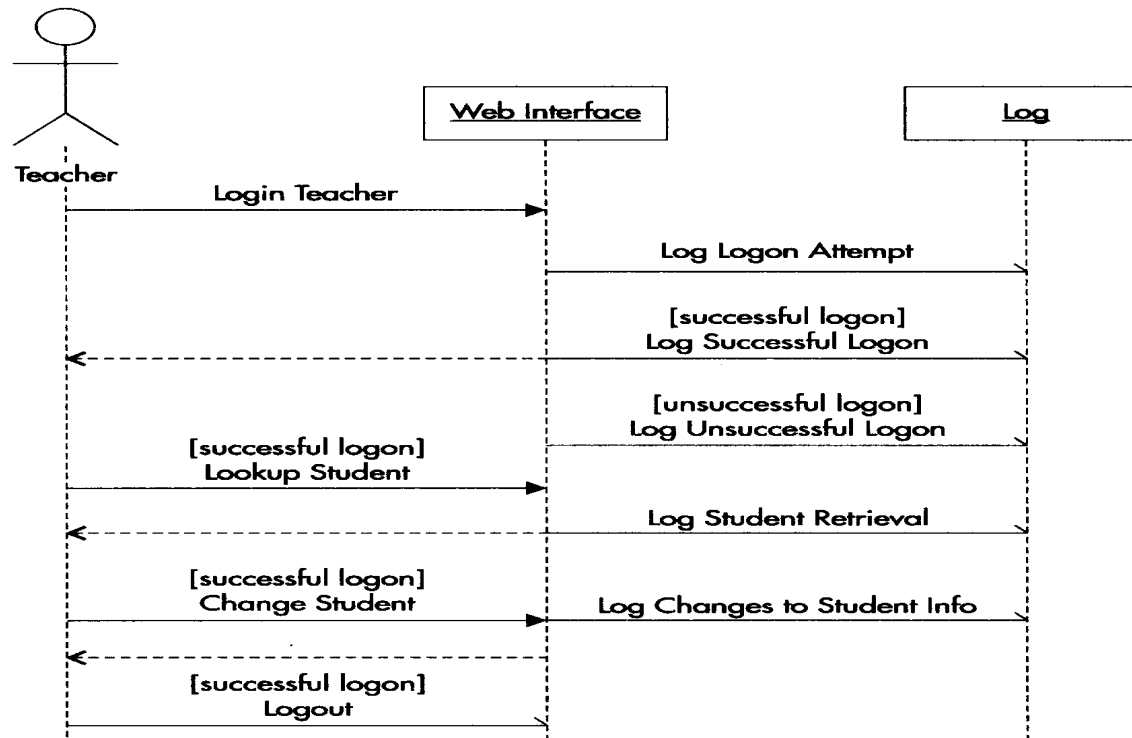
同步消息

- 1 **同步消息** (synchronous message) 代表一个操作调用的控制流。同步消息的发送者把控制传递给消息的接收者，然后暂停活动，等待消息接收者的应答，收到应答后才继续自己的操作。



异步消息

- 异步消息 (Asynchronous message) 用于控制流在完成前不需要中断的情况。消息的发送者把控制传递给消息的接收者, 然后继续自己的活动, 不需等待接收者返回信息或控制。
- 下面示例演示了如何在登录文件的情况下使用异步消息。

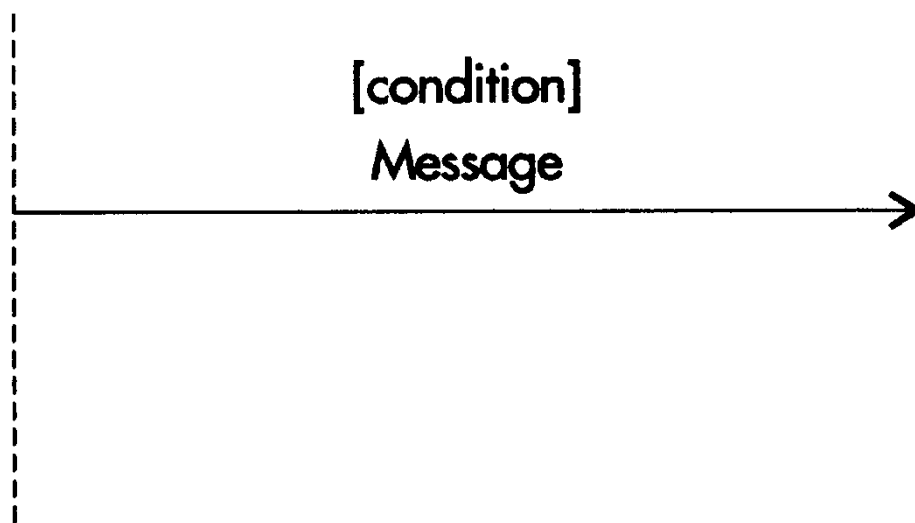


简单消息

- 如果所有的消息都是同步或者异步消息，那么为什么还要简单消息呢？因为有时候我们不关心消息是同步还是异步，此外在高层分析中，有时候没有必要指定一个消息是同步的还是异步的。
- 返回消息（Return message）：一个活动对象收到消息并完成请求的工作后向原对象返回处理结果。

条件

- 消息可以包含**条件**以便限制它们只在满足条件时才能发送。条件显示在消息名称上面的方括号中，如下图所示。



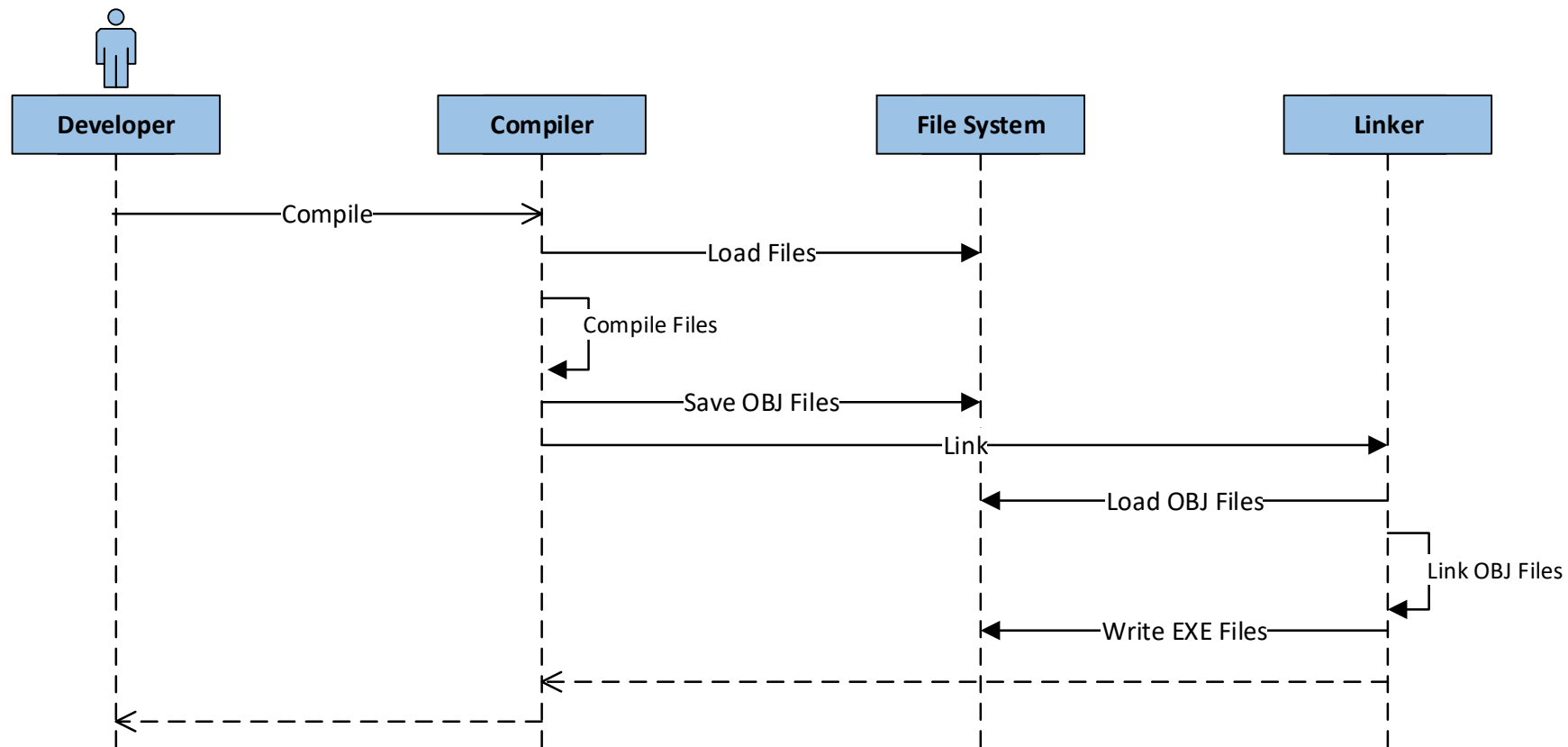
案例：编译程序

- Compile Application用例
- 我们可以创建一个成功编译工作流的顺序图。这个顺序图中有4个活动对象：Developer、Compiler、Linker和File System。Developer是系统的参与者，Compiler是Developer交互的应用程序，Linker是一个用来链接对象文件的独立进程，File System是系统层功能的包装器，用来执行文件的输入和输出例程。

如何使用消息进行通信

- Compile Application用例的顺序图操作：
 - Developer请求Compiler执行编译
 - Compiler请求FileSystem 加载文件
 - Compiler通知自己执行编译
 - Compiler请求FileSystem 保存对象代码
 - Compiler请求Linker链接对象代码
 - Linker请求 FileSystem加载对象代码
 - Liker通知自己执行链接
 - Linker请求FileSystem保存编译的结果

一个成功编译工作流的顺序图



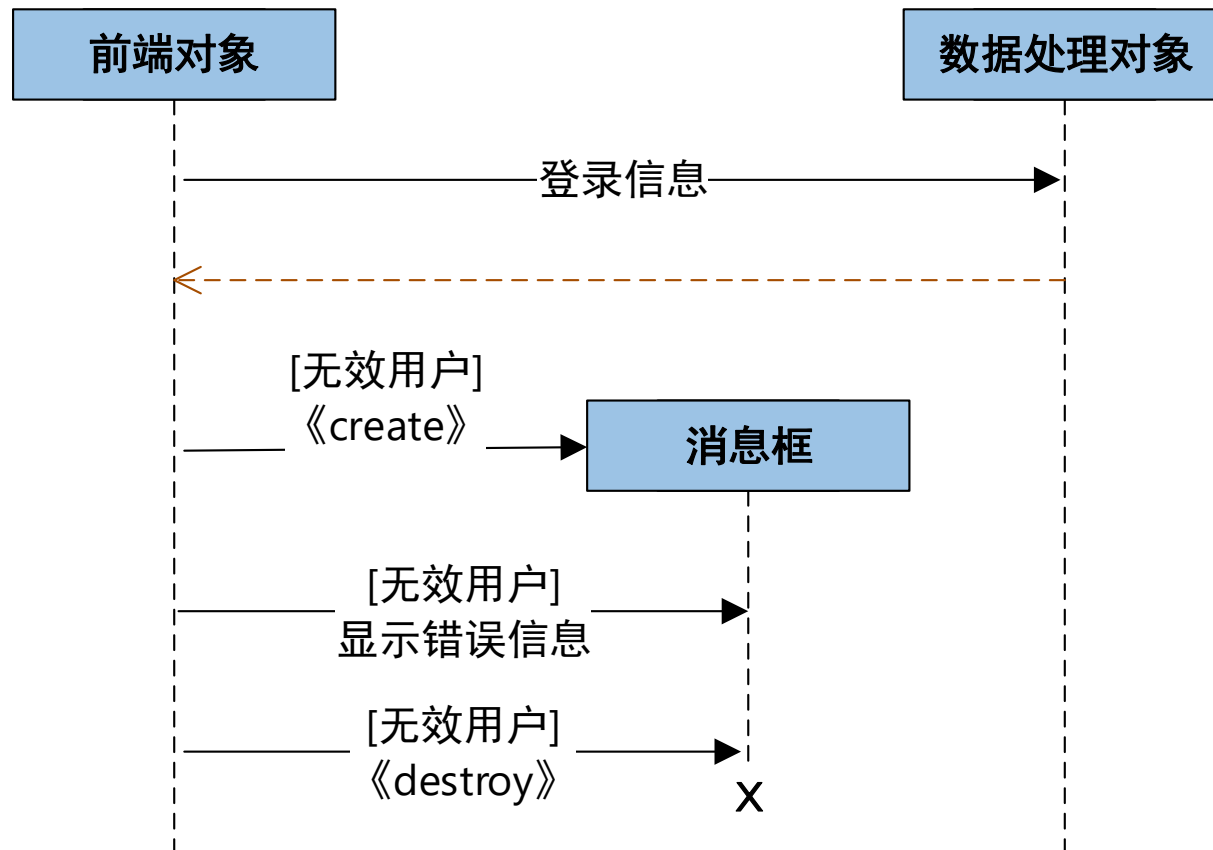
8.3.2 分支和从属

1. 创建与销毁对象

创建对象的标记符如下图中的示例所示。有一个主要步骤用来把“**create**”消息发送给对象实例。**对象创建之后就会具有生命线**，就像顺序图中的任何其他对象一样。现在可以像顺序图中的其他对象那样来使用该对象发送和接收消息。

在处理新创建的对象，或者处理顺序图中的任何其他对象时，都可以发送“**destroys**”消息来**删除对象**。若要想说明某个对象被销毁，需要在被销毁对象的生命线上放一个**X**字符。

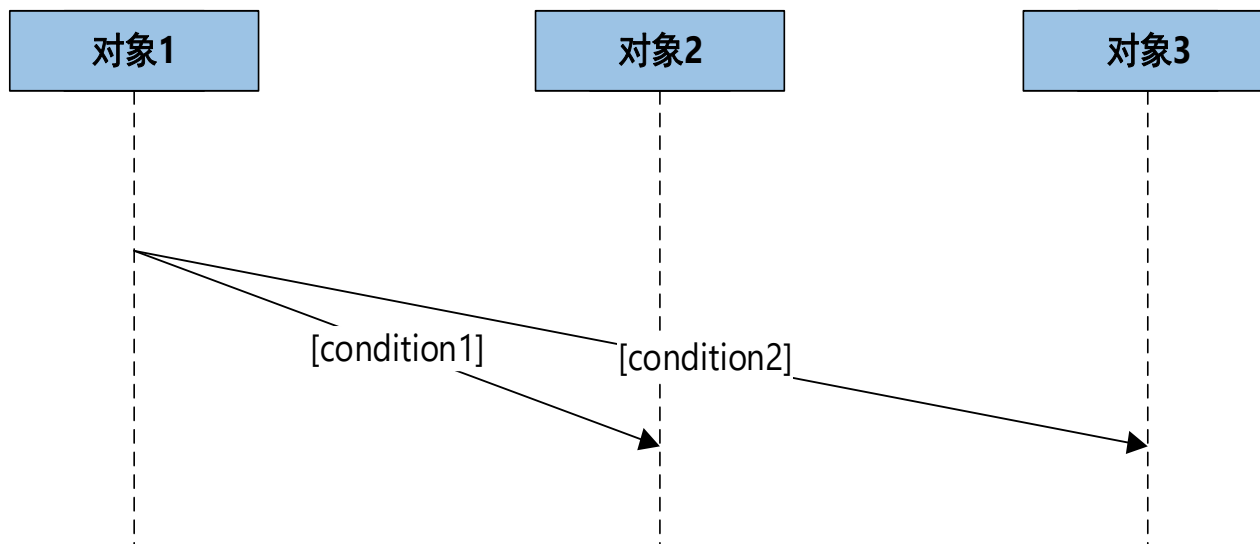
创建与销毁对象



2. 分支和从属流

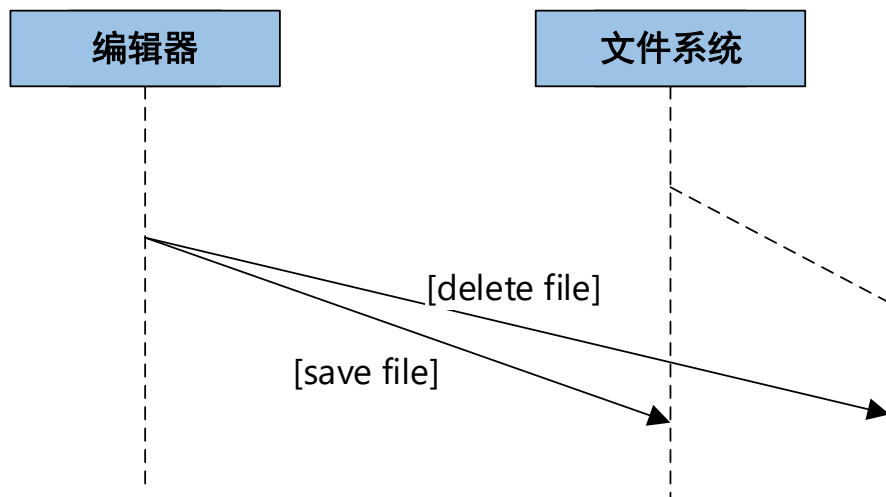
有两种方式来修改顺序图的控制流：分支和从属流。控制流的改变是由于不同的条件导致控制流走向不同的道路。

(1) **分支**允许控制流走向不同的对象，如下图所示。消息的开始位置是相同的，分支消息的结束“高度”也是相等的。这说明在下一步中，**其中之一**将会执行。



从属流

- 从属流指的是从同一点发出多个消息指向同一个对象的不同生命线。
从属流还允许控制流根据条件改变，但是只允许控制流改变为相同对象的另一条生命线分支，如下图所示。
- 在使用文字编辑器时，Editor在用户删除文件或者保存文件时向File System发送消息。显然，File System将会执行两种完全不同的活动，并且每一个工作流都需要独立的生命线，



创建顺序图

创建顺序图包含4项任务：

- 1) 确定需要建模的工作流。
- 2) 从左到右布置对象。
- 3) 添加消息和条件以便创建每一个工作流。
- 4) 绘制总图以便连接各个分图。

例子：建模“教师查询学生分数”

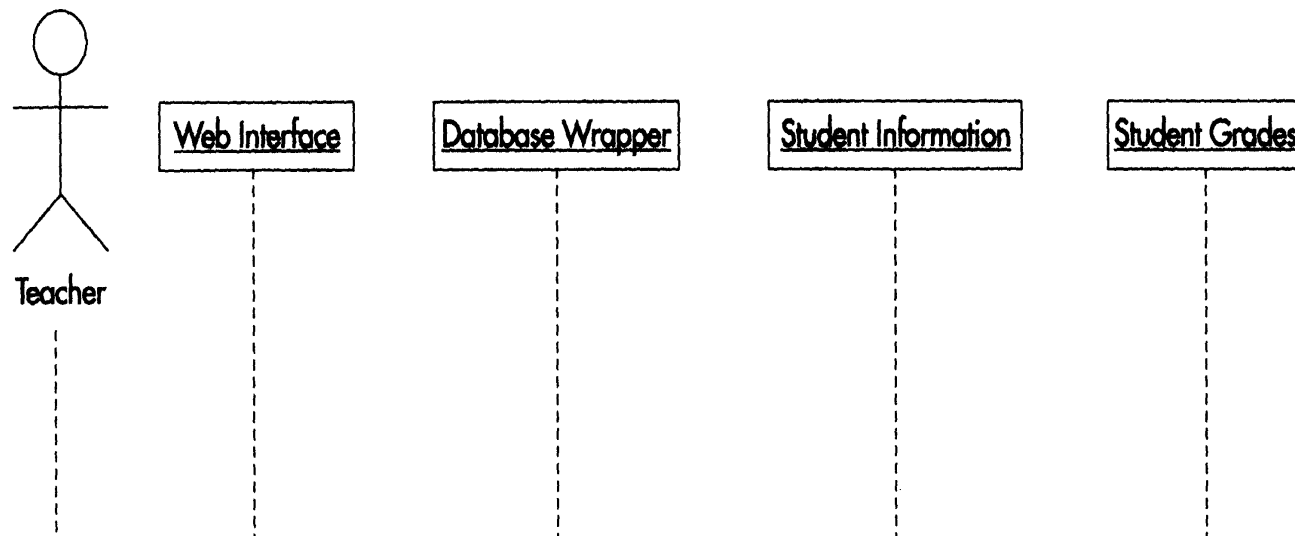
1. 确定工作流

建模顺序图的第一步是确定将要建模的工作流。为此，需要至少标识出3个要建模的工作流：

- 教师成功地检查学生分数
- 教师试图检查某个学生分数，但是该学生在系统中不存在。
- 教师试图检查某个学生分数，但是该学生分数在系统中不存在。

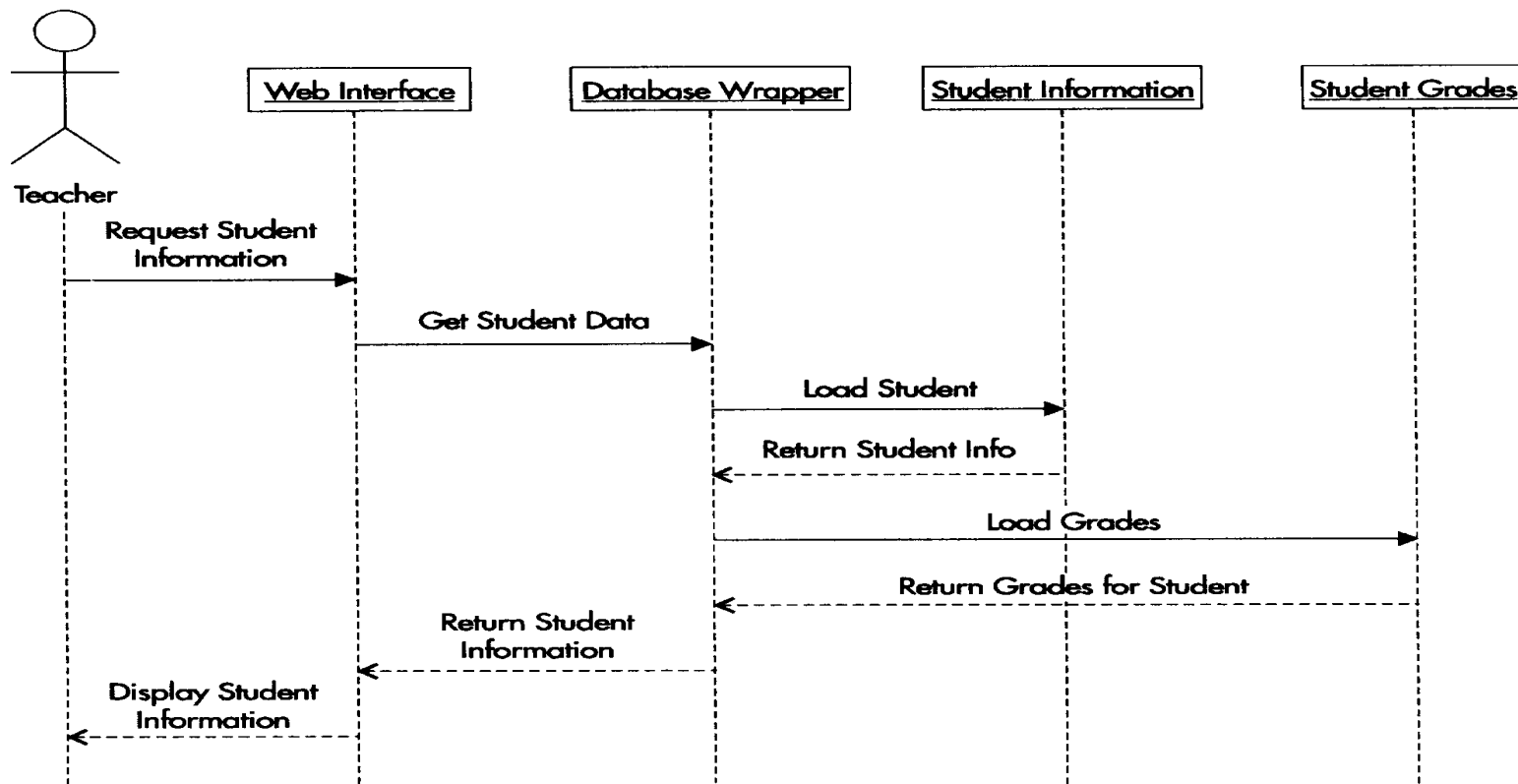
创建顺序图

- 建模顺序图的下一步是从左到右布置所有的参与者和对象，包含要添加消息的对象生命线，如下图所示。



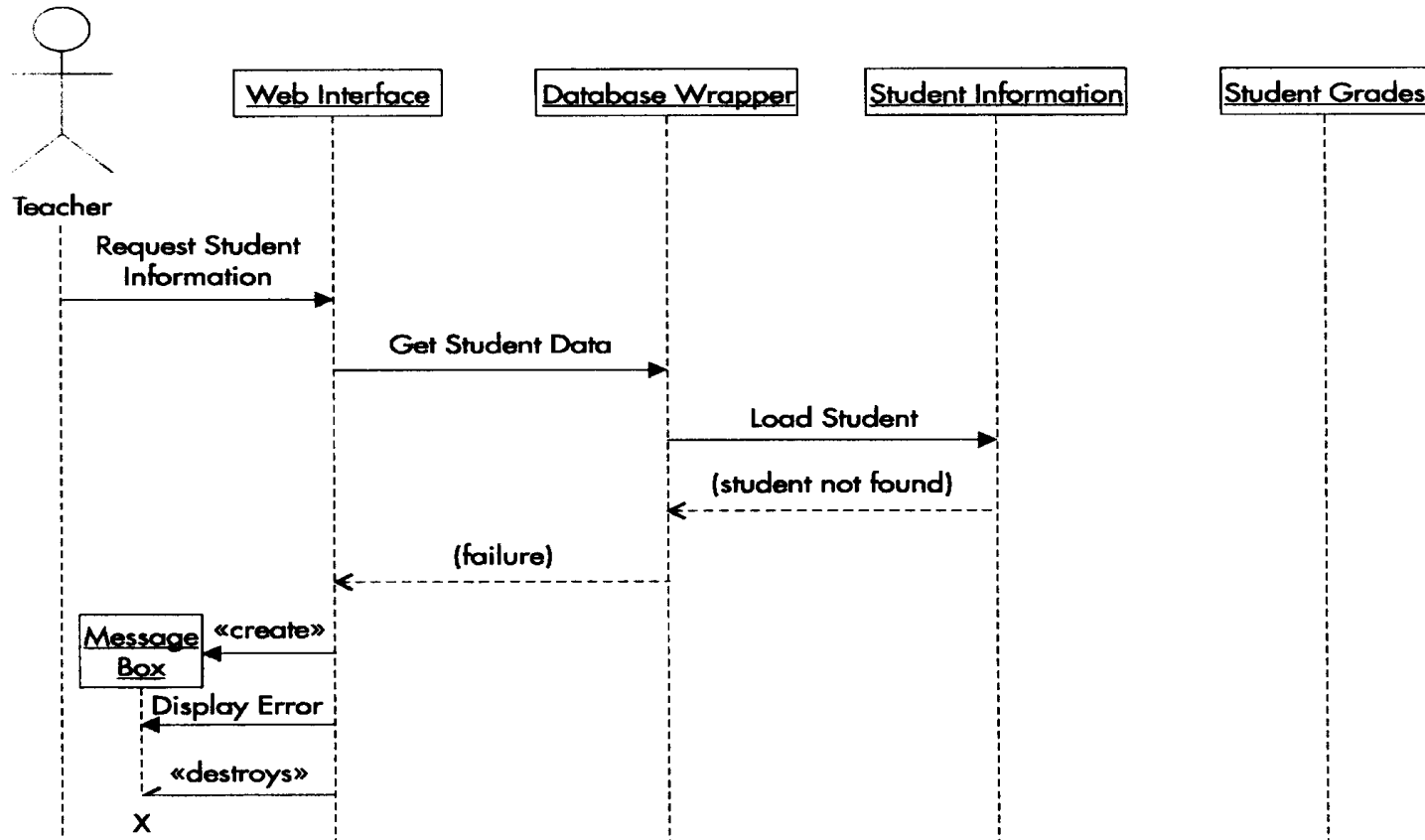
创建顺序图

- 接下来，对每一个工作流作为独立的顺序图建模。从基本的工作流开始，它是没有出错条件，并且需要最少决策的工作流。在本例中，基本工作流是教师成功地检查某个学生的分数，如下图所示。



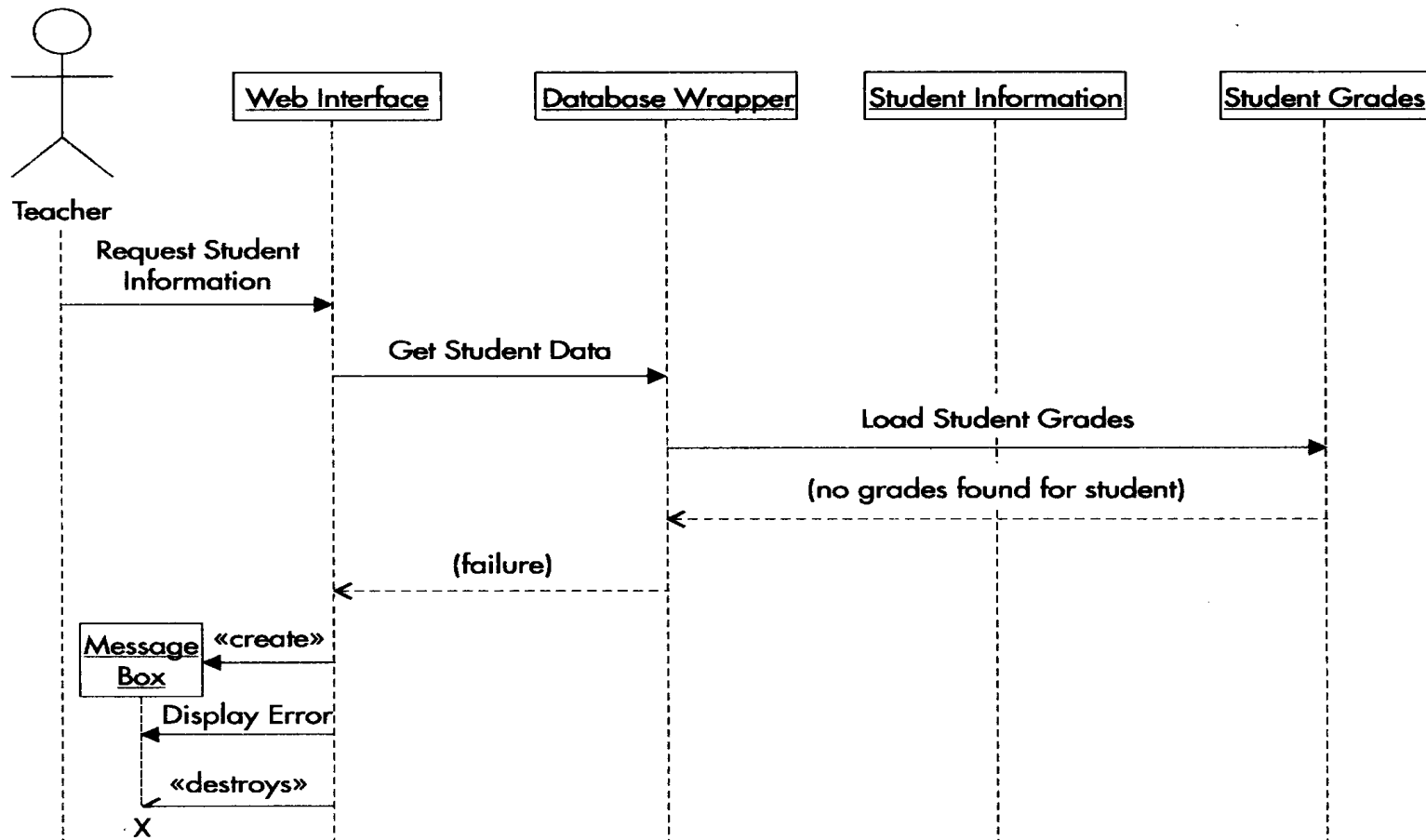
创建顺序图

- 注意选择适当的消息类型（异步、同步、简单和返回）。接下来以独立的顺序图建模从属工作流。此处只建模否定的条件，如下图所示。



创建顺序图

- 学生没分数，注意使用条件来指示在什么时候发送什么消息，如下图所示。



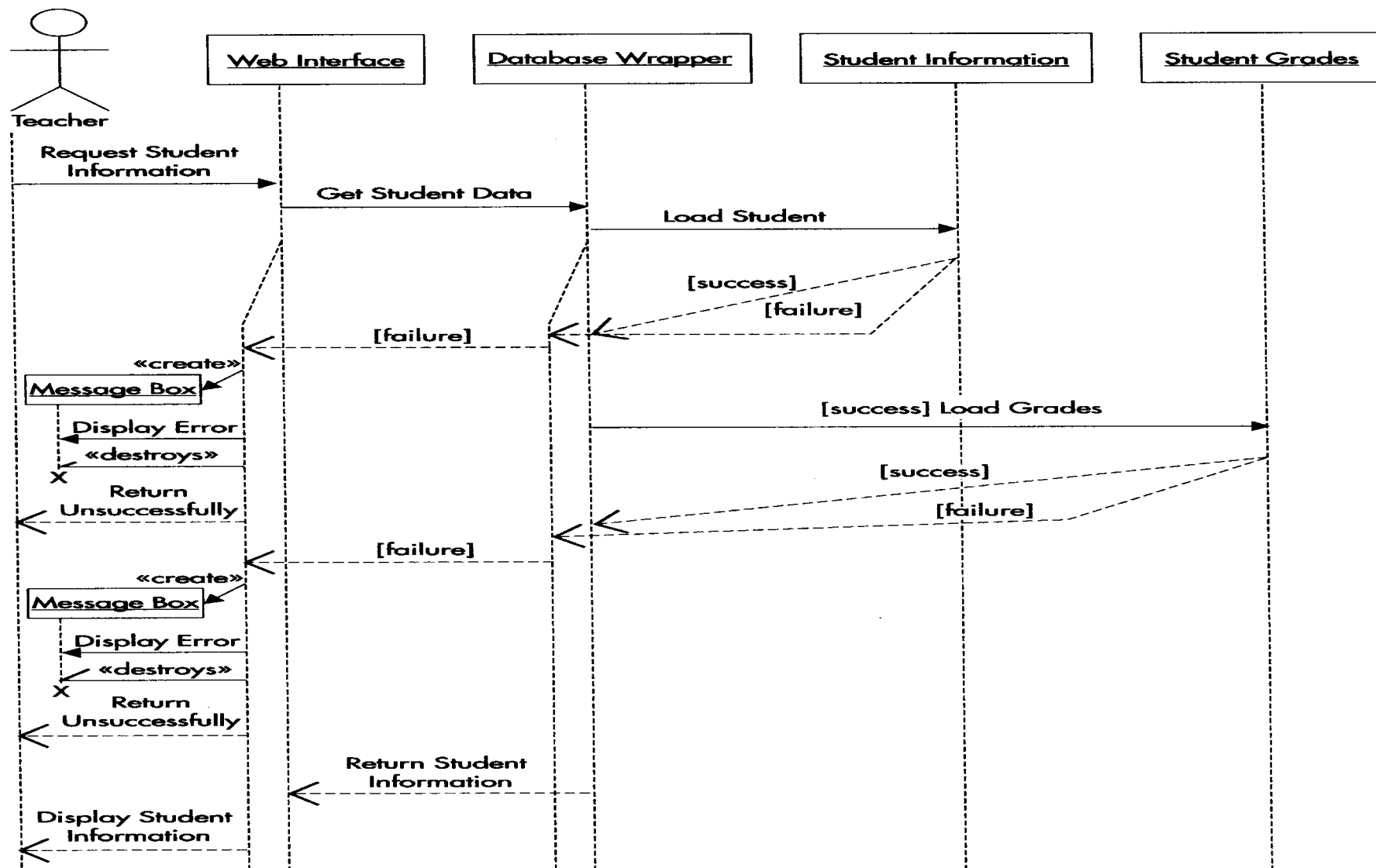
绘制总图

4. 绘制总图

建模顺序图的最后一步是**把所有独立的工作流连接为一个总图**，如下图所示。

在此阶段，如果觉得前面的消息和交互对于当前的顺序图过于详细，可以让它们更加泛化一些，但是在软件建模的下一个阶段，就会觉得初始的各个顺序图越详细越好。

绘制总图

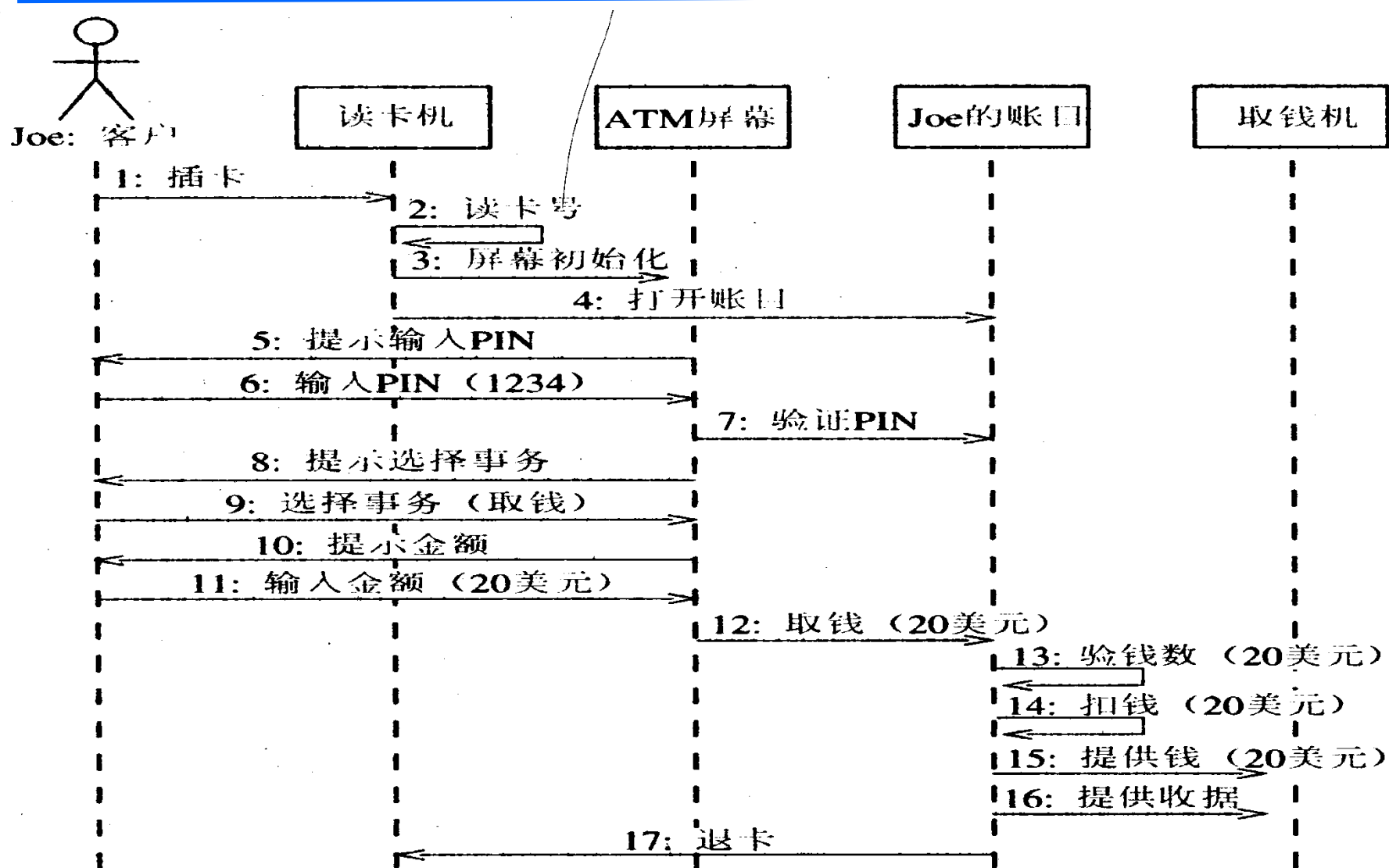


练习

练习：建模一个ATM取款机取款的过程顺序图

对象：读卡机、ATM屏幕、账户、出钱机

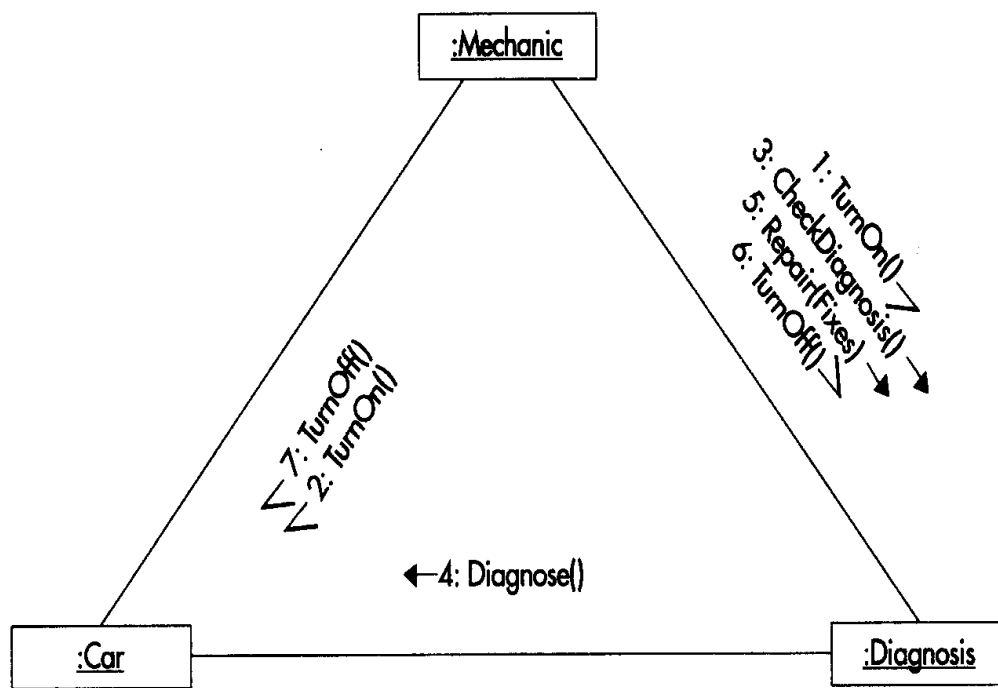
练习



8.3.3 协作图

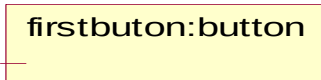
一、协作图

协作图描述系统的行为是如何由系统的成分合作实现的。要想使由类构成的系统具有功能，**这些类的实例（对象）需要彼此通信和交互**。换句话说，它们需要协作。



协作图

● 协作图中的事物及解释

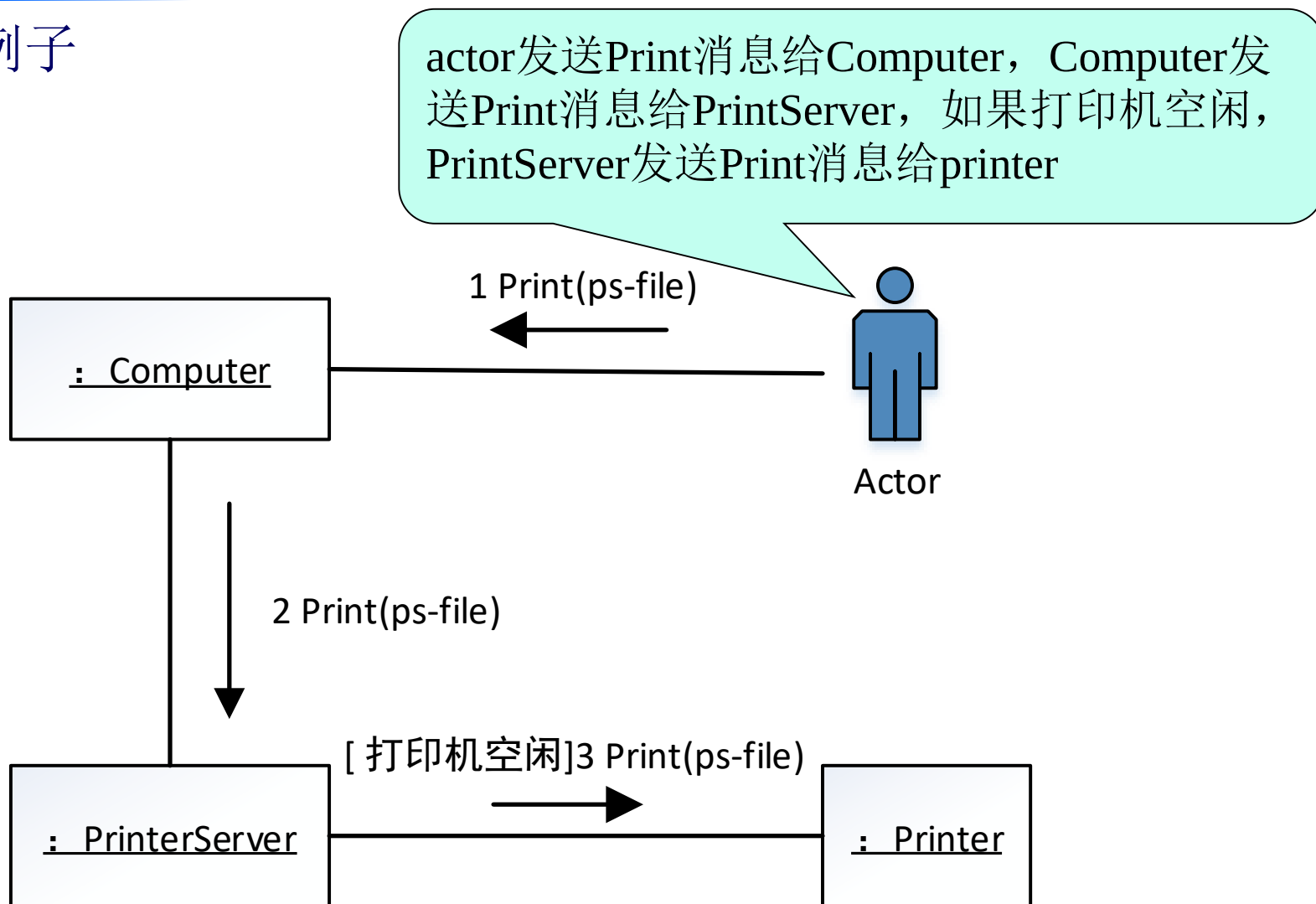
事物名称	解释	图
参与者	发出主动操作的对象，负责发送初始消息，启动一个操作。	 Actor
对象	对象是类的实例，负责发送和接收消息，冒号前为对象名，冒号后为类名。	
消息流 (由箭头和标签组成)	箭头指示消息的流向，从消息的发出者指向接收者。标签对消息作说明，其中， 顺序号 指出消息的发生顺序，并且指明了消息的嵌套关系；冒号后面是消息的名字。	

● 协作图中的关系及解释

关系名称	解释	关系实例
链接	用线条来表示链接，链接表示两个对象共享一个消息，位于对象之间或参与者与对象之间	

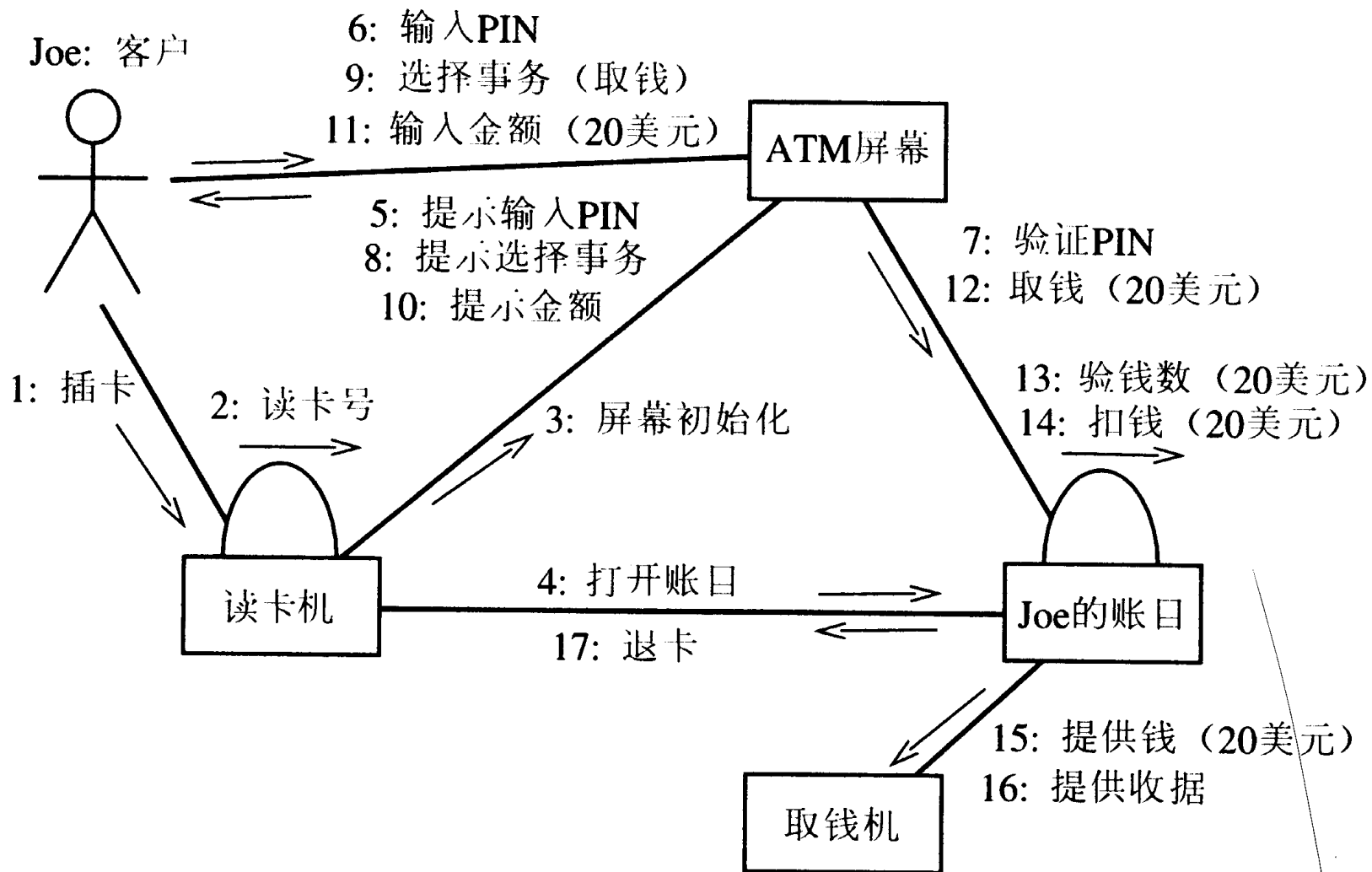
协作图例子

- 协作图例子



协作图例子

取款机取钱的协作图



序列图和协作图区别

序列图和协作图区别

- 序列图和协作图都可以表示对象间的交互关系，但它们的侧重点不同。
- 序列图用消息的几何排列关系来表达对象间交互消息的先后时间顺序。
- 而协作图则建模对象（或角色）间的通信关系。一般适合大于2个参与者的情况。

8.4 实现图

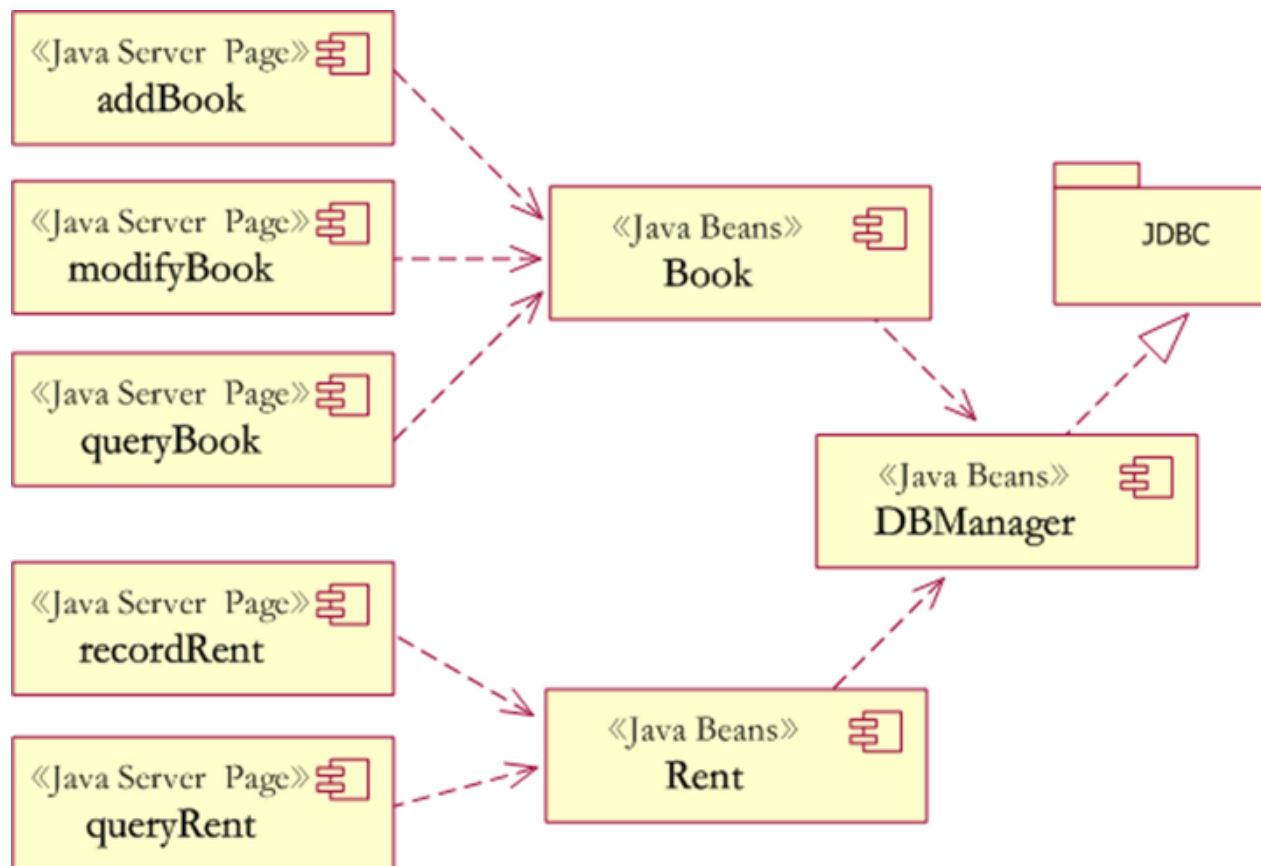
8.4.1 构件图

- 为了实现基于构件（模块）软件开发的设计思想，我们必须将系统划分为若干个可管理的子系统，再把子系统中的类用接口进行封装，以便组成构件内部高内聚 (High Cohesion)、构件之间松耦合 (Loose Coupling) 的结构。

构件图

- 构件图为系统中的构件建模
- 构件建模的目标是把系统中的类分布到更大的内聚的构件中，显示系统构件间的结构关系。
- 从宏观的角度上，构件图把软件看作多个独立构件组装而成的集合，每个构件可以被实现相同接口的其它构件替换。
- 构件图的三个元素是构件、接口和构件之间关系。

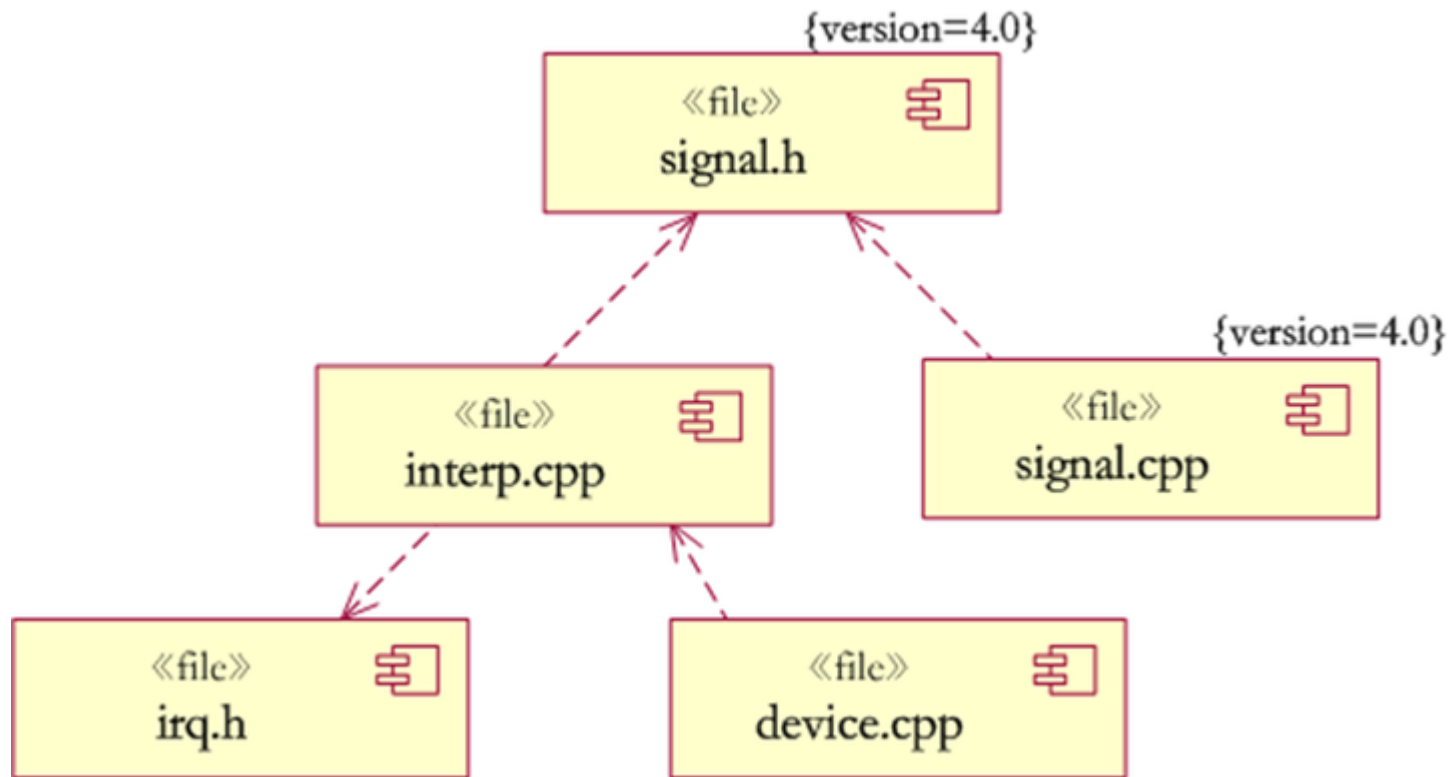
构件图



构件图的作用

- 构件图从软件架构的角度来描述一个系统的主要功能，如系统分成几个子系统，每个子系统包括哪些类、包和构件，它们之间的关系以及它们分配到哪些节点上等。
- 从整体上了解系统的所有物理部件，构件图显示了被开发系统所包含的构件之间的依赖关系。

构件图



构件图

● 构件图的组成



构件

- 在UML2.0中，构件被认为是在一个系统或子系统中的一个独立的封装单位，构件通过一系列的接口对外界提供功能。
- 构件的类型：
 - 1. 部署构件，如dll文件、exe文件、com+对象、corba对象、ejb、动态web页、数据库表等；
 - 2. 工作产品构件，如源代码文件、数据文件等，用来产生部署构件；
 - 3. 执行构件，指系统执行后产生的构件。

构件与类

- 从构件的定义上看，构件和类十分相似，事实也是如此。但也存在着一些明显的不同，下面是构件与类的区别：
 - （1）类表示是对实体的抽象，而构件是对存在于计算机中的物理部件的抽象。也就是说，构件是可以部署的，而类不能部署。
 - （2）构件属于软件模块，而非逻辑模块，与类相比，它们处于不同的抽象级别。甚至可以说，构件就是由一组类通过协作完成的。
 - （3）类可以直接拥有操作和属性，而构件仅拥有可以通过其接口访问的操作。

构件图--接口

- 假如有一个类要使用构件中的某个类的具体的某个方法，但当这个具体的方法发生变化时，那么该类就不能应用构件中的相应内容了
- 在构件图中，构件可以通过其他组件的接口来使用其他构件中定义的操作。
- 通过使用命名的接口，可以避免在系统中各个构件之间直接发生依赖关系，有利于构件的替换。
- 应用接口，可以隐藏具体的实现细节，这样，构件中的内容可以任意变化，而接口却是相对固定的

构件图—接口类型

- 构件向外部展现两种接口：
 - ✓ 供接口
 - ✓ 需接口
- 供接口表示构件为客户提供的功能，它告知用户构件如何被使用。构件至少要有一个供接口
- 需接口表示为了使构件工作，构件必须要从其他服务中所获得的功能。需接口表示该接口是构件的成员变量或构件中类的成员变量

构件图--供接口和需接口

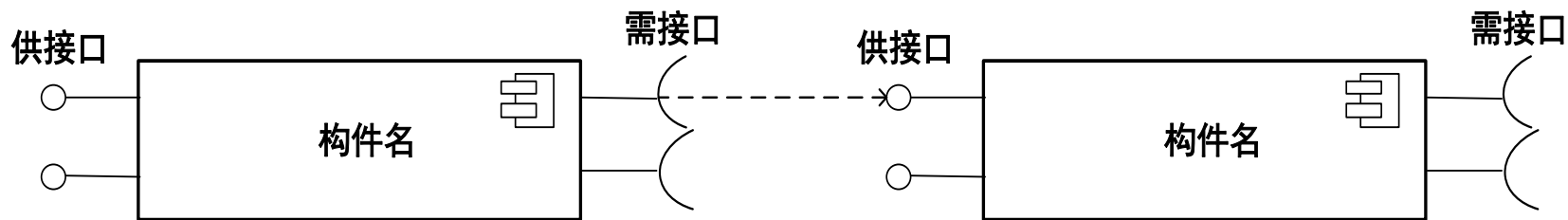
- 供接口用棒棒糖式的图形表示，由一个封闭的圆形与一条直线组成
- 需接口用插座式的图形表示，由一个半圆与一条直线组成



构件图--构件关系

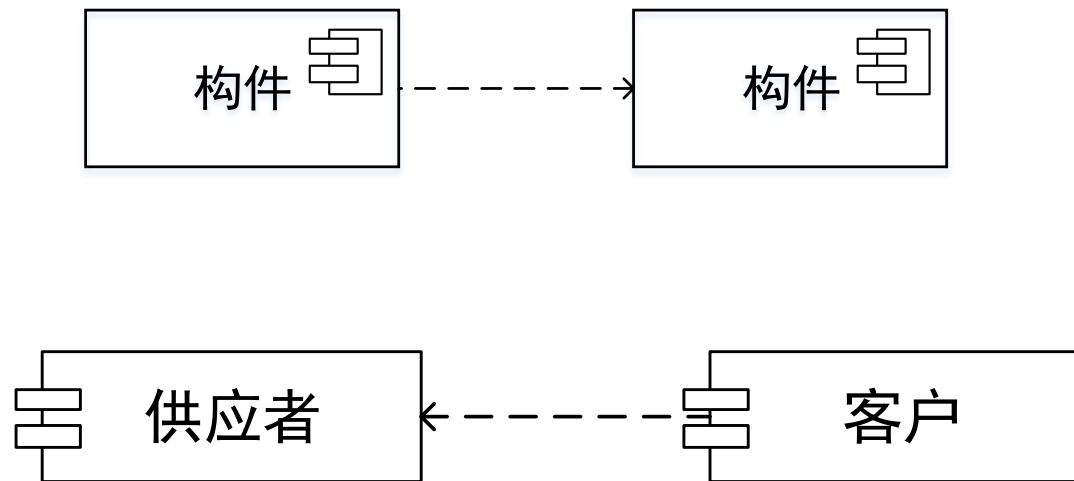
如果一个构件有一个需接口，则表示它需要另一个构件或者类来为它提供服务

- 为了表达构件与其他构件间的关系，供接口与需接口之间用一个表示**依赖的箭头**（即虚线加一个开箭头）连接起来，**该箭头从需接口引出，指向服务供应者提供的供接口**



构建间的关系

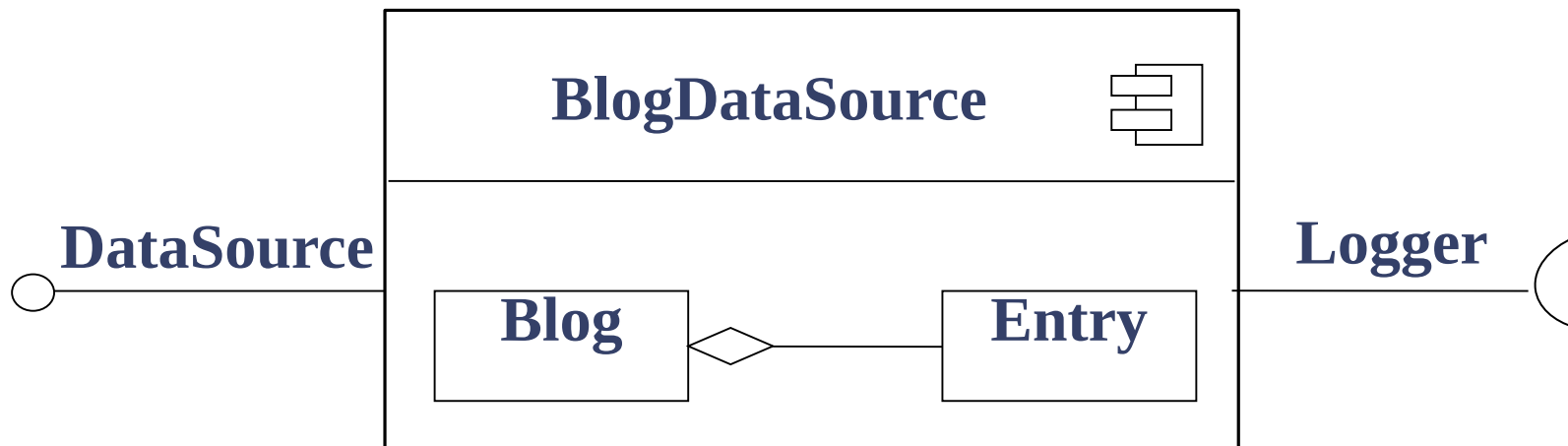
- 更简单的，可以忽略构件间的供接口和需接口，而直接在构件间画上依赖关系，如图所示。



构件图--实现构建的类

实现构件的类

- 构件需要包含和使用一些类来实施它的功能，这些类实现了这个构件
- 可以在构件中画出这些类和类间的关系



实现构件的类